The Effects of Recovery and Workload on Player Physical Performance at the FIFA World Cup

2026-09-28

```
library(dplyr)
```

```
##
## Attaching package: 'dplyr'
```

```
## The following objects are masked from 'package:stats':
##
##     filter, lag
```

```
## The following objects are masked from 'package:base':
##
##     intersect, setdiff, setequal, union
```

```
library(ggplot2)
library(psych)
```

```
##
## Attaching package: 'psych'
```

```
## The following objects are masked from 'package:ggplot2':
##
##     %+%, alpha
```

```
library(scales)
```

```
##
## Attaching package: 'scales'
```

```
## The following objects are masked from 'package:psych':
##
##     alpha, rescale
```

```
library(splines)
library(lme4)
```

```
## Loading required package: Matrix
```

```
library(lmerTest)
```

```
##
## Attaching package: 'lmerTest'
```

```
## The following object is masked from 'package:lme4':
##
##     lmer
```

```
## The following object is masked from 'package:stats':
##
##     step
```

1. Data Processing

```r
# Load raw data
df <- read.csv("raw_data.csv")
nrow(df)
```

```
## [1] 5287
```

```r
# Variables whose previous-match values we want to retain
prev_vars <- c(
  "minutes_played",
  "total_distance_m"
)

# Add previous-match information
df <- df %>%
  mutate(
    # Convert match_date to Date
    match_date = as.Date(match_date)
  ) %>%

  # Drop rows where the player never actually appeared (0 distance covered), so they aren't treated as
  filter(total_distance_m > 0) %>%

  # Put each player's appearances in chronological order
  arrange(
    tournament_year,
    player_tournament_id,
    match_date,
    fifa_match_number
  ) %>%

  # Previous matches must be computed separately within each tournament/player
  group_by(tournament_year, player_tournament_id) %>%

  mutate(
    # Match metadata from previous appearance
    prev_match_id = lag(match_id),
    prev_match_date = lag(match_date),

    # Days since previous appearance
    recovery_days = as.integer(match_date - prev_match_date),

    # Previous-match statistics
    across(
      all_of(prev_vars),
      lag,
      .names = "prev_{.col}"
    )
  ) %>%

  ungroup()
```

```r
df <- df %>%
  # Filter out rows where player played less than 10 minutes
```

```r
  filter(
    !(is.na(minutes_played) | minutes_played < 10)
  ) %>%

  # Normalize statistics by minutes played
  mutate(
    normalized_total_distance_m = total_distance_m / minutes_played,
    normalized_pressing_direct = pressing_direct / minutes_played
  ) %>%

  # Add binned minutes_played column
  mutate(
    binned_minutes_played = cut(minutes_played, breaks = c(0, 45, 89, max(minutes_played)), include.lowe
  )

nrow(df)
```

```
## [1] 4875
```

```r
table(df$binned_minutes_played)
```

```
##
##   [0,45]  (45,89] (89,120]
##     1233     1372     2270
```

```r
head(df)
```

```
## # A tibble: 6 x 63
##   tournament_year match_id  fifa_match_number stage  stage_type group match_date
##             <int> <chr>                 <int> <chr>  <chr>      <chr> <date>
## 1            2022 2022_M008                 8 Group~ group      "C"   2022-11-22
## 2            2022 2022_M024                24 Group~ group      "C"   2022-11-26
## 3            2022 2022_M039                39 Group~ group      "C"   2022-11-30
## 4            2022 2022_M050                50 Round~ knockout   ""    2022-12-03
## 5            2022 2022_M057                57 Quart~ knockout   ""    2022-12-09
## 6            2022 2022_M064                64 Final  knockout   ""    2022-12-18
## # i 56 more variables: kickoff_time_local <chr>, stadium <chr>, team <chr>,
## #   opponent <chr>, team_goals <int>, opponent_goals <int>,
## #   team_tournament_id <chr>, player_name <chr>, player_tournament_id <chr>,
## #   shirt_number <int>, position <chr>, starter <int>, sub_on_minute <int>,
## #   sub_off_minute <int>, sent_off_minute <int>, match_duration_min <int>,
## #   minutes_played <int>, total_distance_m <dbl>, zone1_distance_m <dbl>,
## #   zone2_distance_m <dbl>, zone3_distance_m <dbl>, zone4_distance_m <dbl>, ...
```

```r
str(df)
```

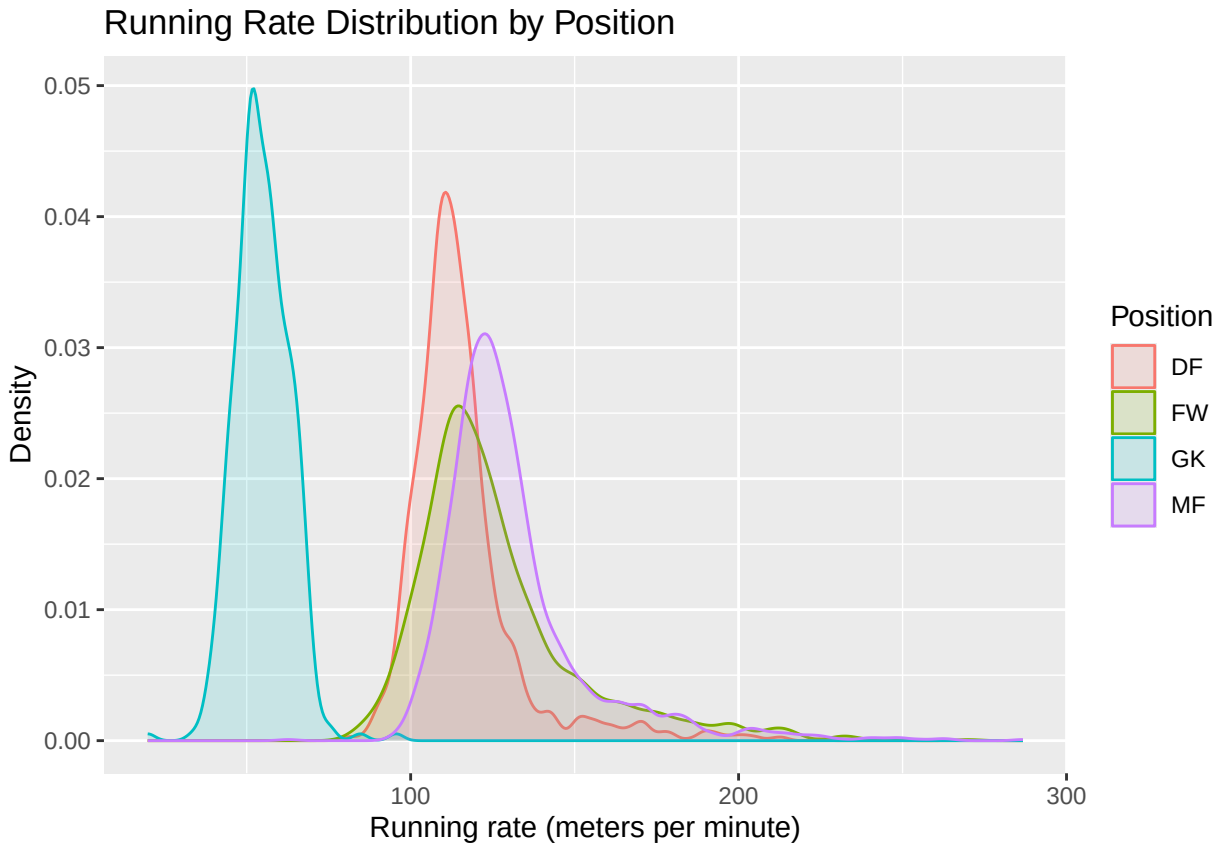
```
## tibble [4,875 x 63] (S3: tbl_df/tbl/data.frame)
##  $ tournament_year            : int [1:4875] 2022 2022 2022 2022 2022 2022 2022 2022 2022 2022 ...
##  $ match_id                   : chr [1:4875] "2022_M008" "2022_M024" "2022_M039" "2022_M050" ...
##  $ fifa_match_number          : int [1:4875] 8 24 39 50 57 64 8 24 39 50 ...
##  $ stage                      : chr [1:4875] "Group Stage" "Group Stage" "Group Stage" "Round of 16"
##  $ stage_type                 : chr [1:4875] "group" "group" "group" "knockout" ...
##  $ group                      : chr [1:4875] "C" "C" "C" "" ...
##  $ match_date                 : Date[1:4875], format: "2022-11-22" "2022-11-26" ...
##  $ kickoff_time_local         : chr [1:4875] "13:00" "22:00" "22:00" "22:00" ...
##  $ stadium                    : chr [1:4875] "Lusail Stadium" "Lusail Stadium" "Stadium 974" "Ahmad
```

```
##  $ team                       : chr [1:4875] "Argentina" "Argentina" "Argentina" "Argentina" ...
##  $ opponent                   : chr [1:4875] "Saudi Arabia" "Mexico" "Poland" "Australia" ...
##  $ team_goals                 : int [1:4875] 1 2 2 2 2 3 1 2 2 2 ...
##  $ opponent_goals             : int [1:4875] 2 0 0 1 2 3 2 0 0 1 ...
##  $ team_tournament_id         : chr [1:4875] "2022_ARGENTINA" "2022_ARGENTINA" "2022_ARGENTINA" "202
##  $ player_name                : chr [1:4875] "ACUNA Marcos" "ACUNA Marcos" "ACUNA Marcos" "ACUNA Mar
##  $ player_tournament_id       : chr [1:4875] "2022_ARGENTINA_ACUNA_MARCOS_8" "2022_ARGENTINA_ACUNA_M
##  $ shirt_number               : int [1:4875] 8 8 8 8 8 8 9 9 9 9 ...
##  $ position                   : chr [1:4875] "MF" "MF" "MF" "MF" ...
##  $ starter                    : int [1:4875] 0 1 1 1 1 0 0 0 1 1 ...
##  $ sub_on_minute              : int [1:4875] 71 NA NA NA NA 64 59 63 NA NA ...
##  $ sub_off_minute             : int [1:4875] NA NA 59 72 78 NA NA NA 79 72 ...
##  $ sent_off_minute            : int [1:4875] NA NA NA NA NA NA NA NA NA NA ...
##  $ match_duration_min         : int [1:4875] 90 90 90 90 120 120 90 90 90 90 ...
##  $ minutes_played             : int [1:4875] 19 90 59 72 78 56 31 27 79 72 ...
##  $ total_distance_m           : num [1:4875] 3237 9991 6790 7821 8271 ...
##  $ zone1_distance_m           : num [1:4875] 1220 3888 2330 2693 3164 ...
##  $ zone2_distance_m           : num [1:4875] 1302 4326 3036 3625 3417 ...
##  $ zone3_distance_m           : num [1:4875] 476 974 847 881 1015 ...
##  $ zone4_distance_m           : num [1:4875] 171 552 452 436 512 ...
##  $ zone5_distance_m           : num [1:4875] 69 251 126 186 163 ...
##  $ high_speed_runs            : int [1:4875] 39 101 73 88 86 95 51 51 114 103 ...
##  $ sprints                    : int [1:4875] 16 47 29 34 38 36 29 20 52 50 ...
##  $ top_speed_kmh              : num [1:4875] 31.9 29.6 30.7 29.5 31.6 30 31.6 30 32.3 32.4 ...
##  $ high_speed_runs_definition : chr [1:4875] "Not specified on table" "Not specified on table" "Zone
##  $ sprints_definition         : chr [1:4875] "Not specified on table" "Not specified on table" "Zone
##  $ tackles_made               : int [1:4875] 0 4 1 3 4 4 1 1 1 1 ...
##  $ tackles_won                : int [1:4875] 0 2 0 1 2 2 1 0 1 1 ...
##  $ blocks                     : int [1:4875] 2 4 2 3 4 2 1 1 1 3 ...
##  $ interceptions              : int [1:4875] 0 1 0 0 0 0 0 0 0 0 ...
##  $ pressing_direct            : int [1:4875] 0 8 4 4 4 3 2 2 3 6 ...
##  $ pressing_indirect          : int [1:4875] 3 15 5 13 12 17 7 14 19 31 ...
##  $ aerial_duels_won           : int [1:4875] 1 2 1 2 0 2 1 0 1 0 ...
##  $ physical_duels_won         : int [1:4875] 0 0 0 1 0 0 0 0 0 1 ...
##  $ possession_contests_won    : int [1:4875] 1 4 1 3 0 2 1 0 1 1 ...
##  $ clearances                 : int [1:4875] 3 1 0 1 1 2 0 2 1 0 ...
##  $ loose_ball_receptions      : int [1:4875] 3 4 12 2 3 4 0 2 3 2 ...
##  $ pushing_on                 : int [1:4875] 4 23 8 26 25 22 13 14 29 47 ...
##  $ pushing_on_into_pressing   : int [1:4875] 3 13 4 12 9 15 3 7 11 23 ...
##  $ possession_regains         : int [1:4875] 3 3 2 5 5 3 2 1 1 2 ...
##  $ possession_interrupted     : int [1:4875] 3 0 0 0 3 3 1 4 1 0 ...
##  $ source_pdf                 : chr [1:4875] "M08 ARG v KSA post-match report 1.1.pdf" "report_13301
##  $ lineup_page                : int [1:4875] 2 2 2 2 2 2 2 2 2 2 ...
##  $ physical_page              : int [1:4875] 50 50 52 50 51 51 50 50 52 50 ...
##  $ oop_page                   : int [1:4875] 47 47 49 47 48 48 47 47 49 47 ...
##  $ qc_flag                    : chr [1:4875] "" "" "" "" ...
##  $ prev_match_id              : chr [1:4875] NA "2022_M008" "2022_M024" "2022_M039" ...
##  $ prev_match_date            : Date[1:4875], format: NA "2022-11-22" ...
##  $ recovery_days              : int [1:4875] NA 4 4 3 6 9 NA 4 4 3 ...
##  $ prev_minutes_played        : int [1:4875] NA 19 90 59 72 78 NA 31 27 79 ...
##  $ prev_total_distance_m      : num [1:4875] NA 3237 9991 6790 7821 ...
##  $ normalized_total_distance_m: num [1:4875] 170 111 115 109 106 ...
##  $ normalized_pressing_direct : num [1:4875] 0 0.0889 0.0678 0.0556 0.0513 ...
##  $ binned_minutes_played      : Factor w/ 3 levels "[0,45]","(45,89]",..: 1 3 2 2 2 2 1 1 2 2 ...
```

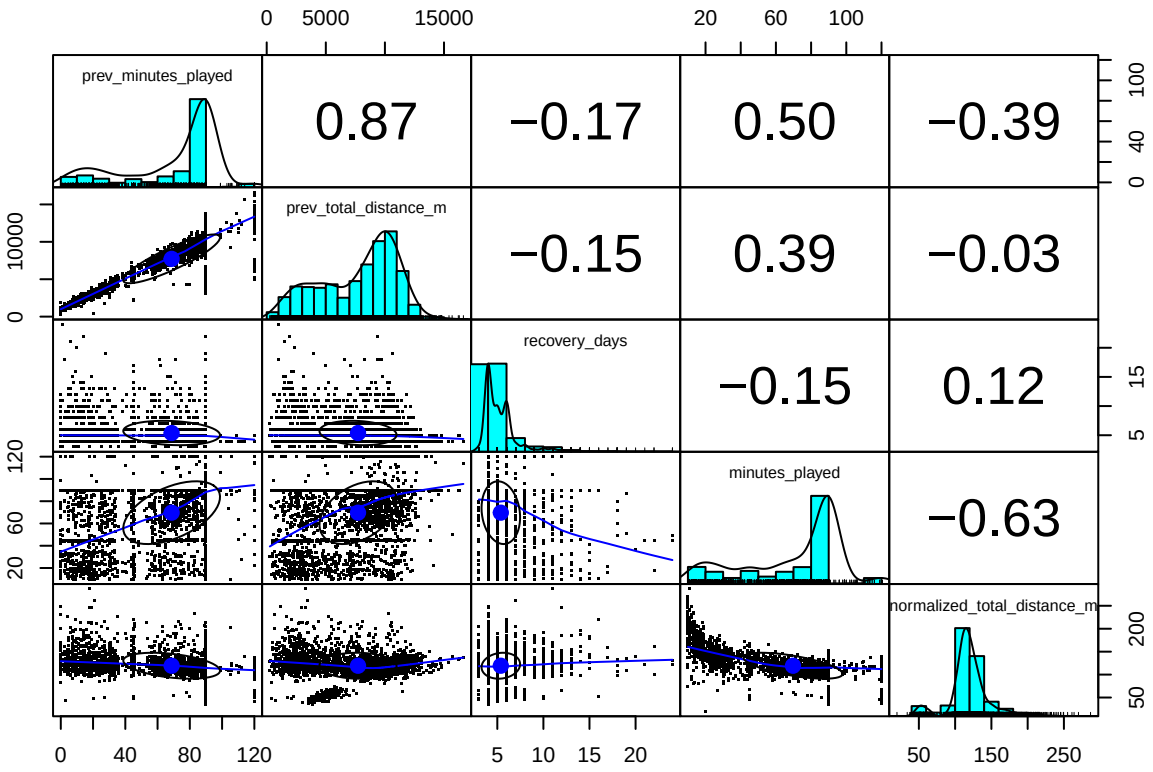
2. Data Visualization

Running Rate

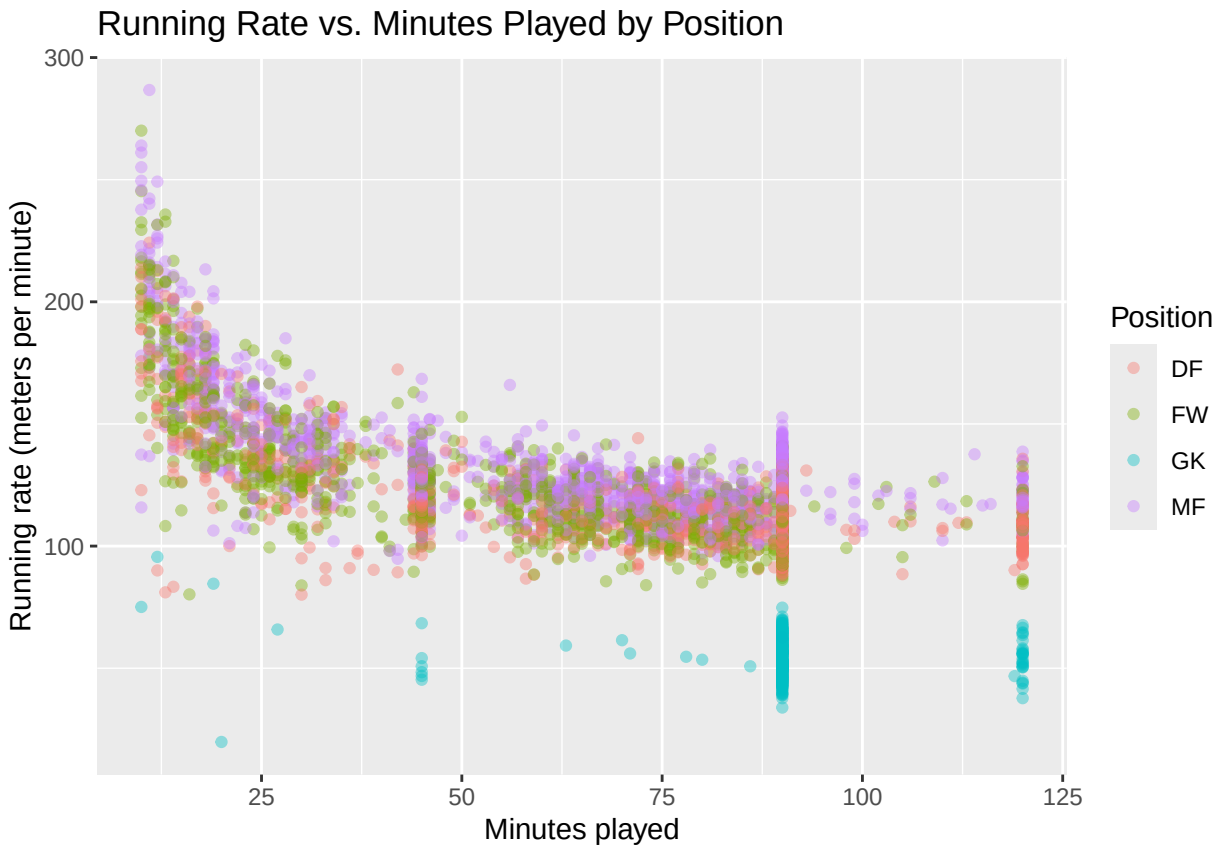
```
# Running Rate Distribution by Position
df %>%
  ggplot(aes(
    x = normalized_total_distance_m,
    color = position,
    fill = position
  )) +
  geom_density(alpha = 0.15) +
  labs(
    title = "Running Rate Distribution by Position",
    x = "Running rate (meters per minute)",
    y = "Density",
    color = "Position",
    fill = "Position"
  )
```




```
pairs.panels(df[, c(
  "prev_minutes_played",
  "prev_total_distance_m",
  "recovery_days",
  "minutes_played",
  "normalized_total_distance_m"
)], gap = 0, pch = ".")
```

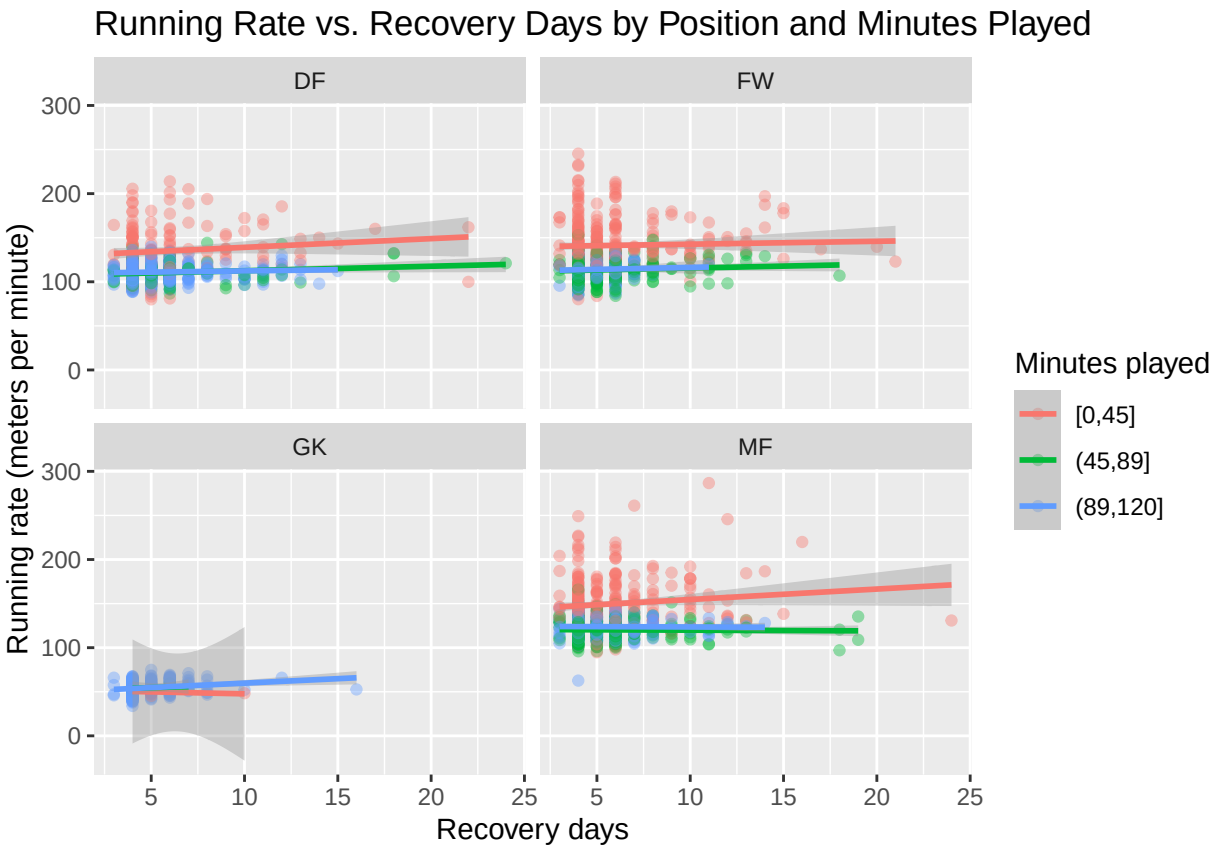



```
# Running Rate vs. Minutes Played by Position
df %>%
  ggplot(aes(
    x = minutes_played,
    y = normalized_total_distance_m,
    color = position
  )) +
  geom_point(alpha = 0.4) +
  labs(
    title = "Running Rate vs. Minutes Played by Position",
    x = "Minutes played",
    y = "Running rate (meters per minute)",
    color = "Position"
  )
```



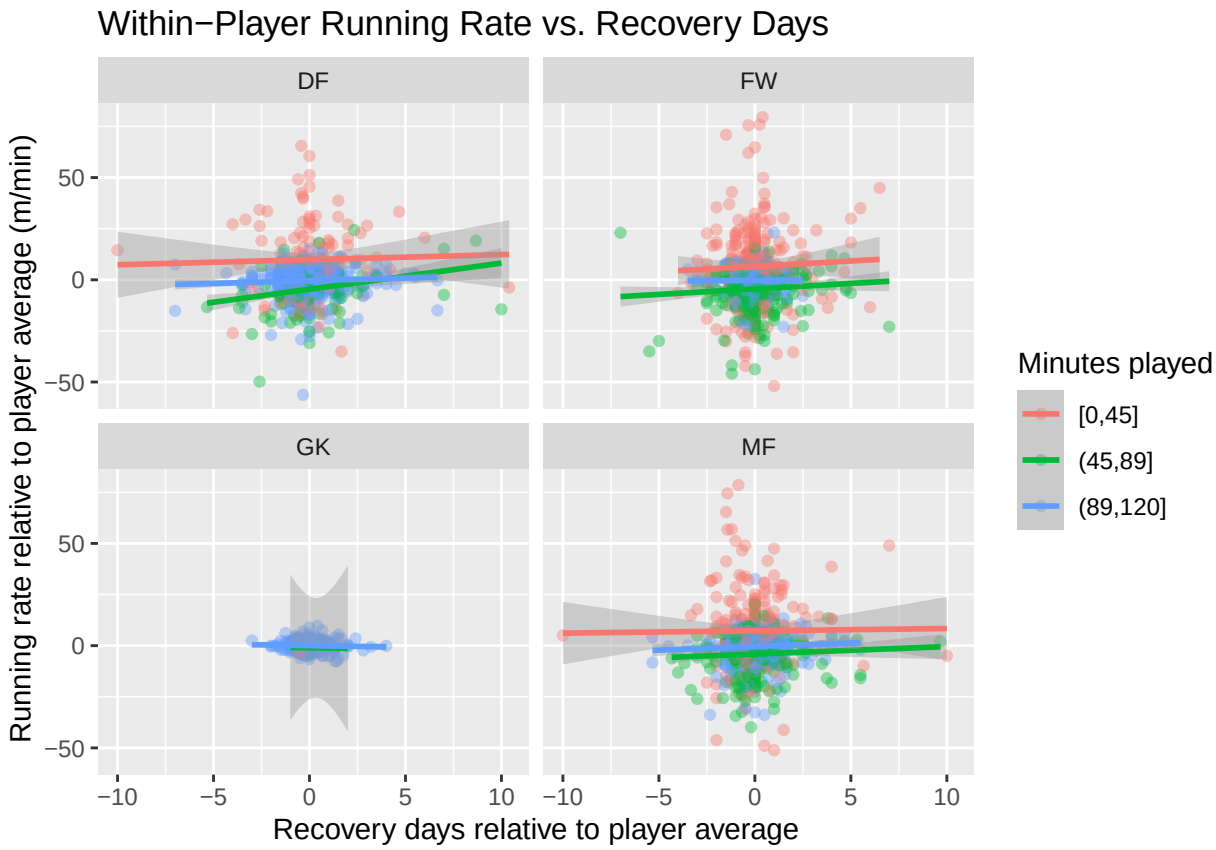

```
# Running Rate vs. Recovery Days by Position and Minutes Played
df %>%
  filter(!is.na(recovery_days)) %>%
  # filter(recovery_days < 15) %>%
  ggplot(aes(
    x = recovery_days,
    y = normalized_total_distance_m,
    color = binned_minutes_played
  )) +
  geom_point(alpha = 0.4) +
  geom_smooth(method = "lm") +
  facet_wrap(~position) +
  labs(
    title = "Running Rate vs. Recovery Days by Position and Minutes Played",
    x = "Recovery days",
    y = "Running rate (meters per minute)",
    color = "Minutes played"
  )
```

```
## `geom_smooth()` using formula = 'y ~ x'
```




```
# Within-Player Running Rate vs. Recovery Days
df %>%
  filter(!is.na(recovery_days)) %>%
  group_by(player_tournament_id) %>%
  filter(n() >= 2) %>%
  mutate(
    recovery_dev = recovery_days - mean(recovery_days),
    running_rate_dev = normalized_total_distance_m - mean(normalized_total_distance_m)
  ) %>%
  ungroup() %>%
  ggplot(aes(
    x = recovery_dev,
    y = running_rate_dev,
    color = binned_minutes_played
  )) +
  geom_point(alpha = 0.4) +
  geom_smooth(method = "lm") +
  facet_wrap(~position) +
  labs(
    title = "Within-Player Running Rate vs. Recovery Days",
    x = "Recovery days relative to player average",
    y = "Running rate relative to player average (m/min)",
    color = "Minutes played"
  )
```

```
## `geom_smooth()` using formula = 'y ~ x'
```



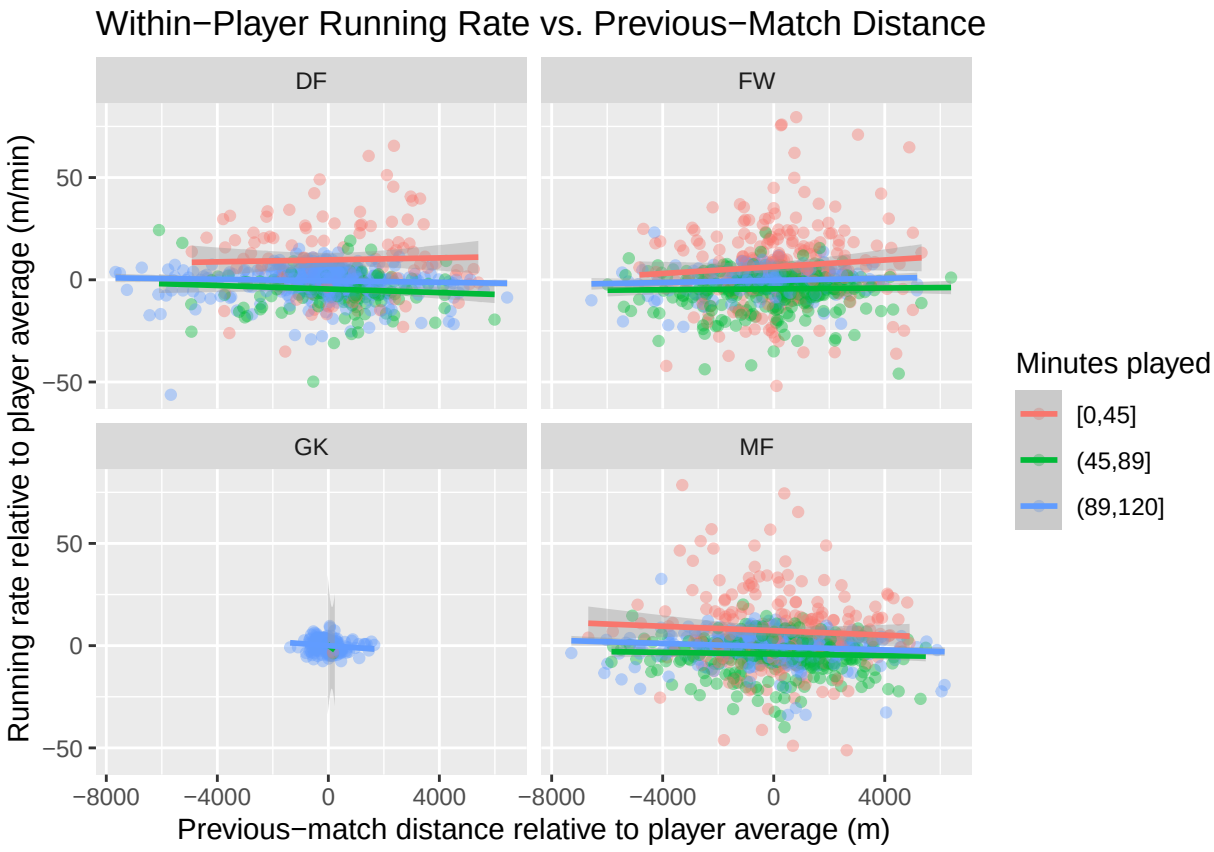

```r
# Running Rate vs. Previous-Match Distance by Position and Minutes Played
df %>%
  filter(!is.na(prev_total_distance_m)) %>%
  ggplot(aes(
    x = prev_total_distance_m,
    y = normalized_total_distance_m,
    color = binned_minutes_played
  )) +
  geom_point(alpha = 0.4) +
  geom_smooth(method = "lm") +
  facet_wrap(~position) +
  labs(
    title = "Running Rate vs. Previous-Match Distance by Position and Minutes Played",
    x = "Previous-match distance (meters)",
    y = "Running rate (meters per minute)",
    color = "Minutes played"
  )
```

```
## `geom_smooth()` using formula = 'y ~ x'
```



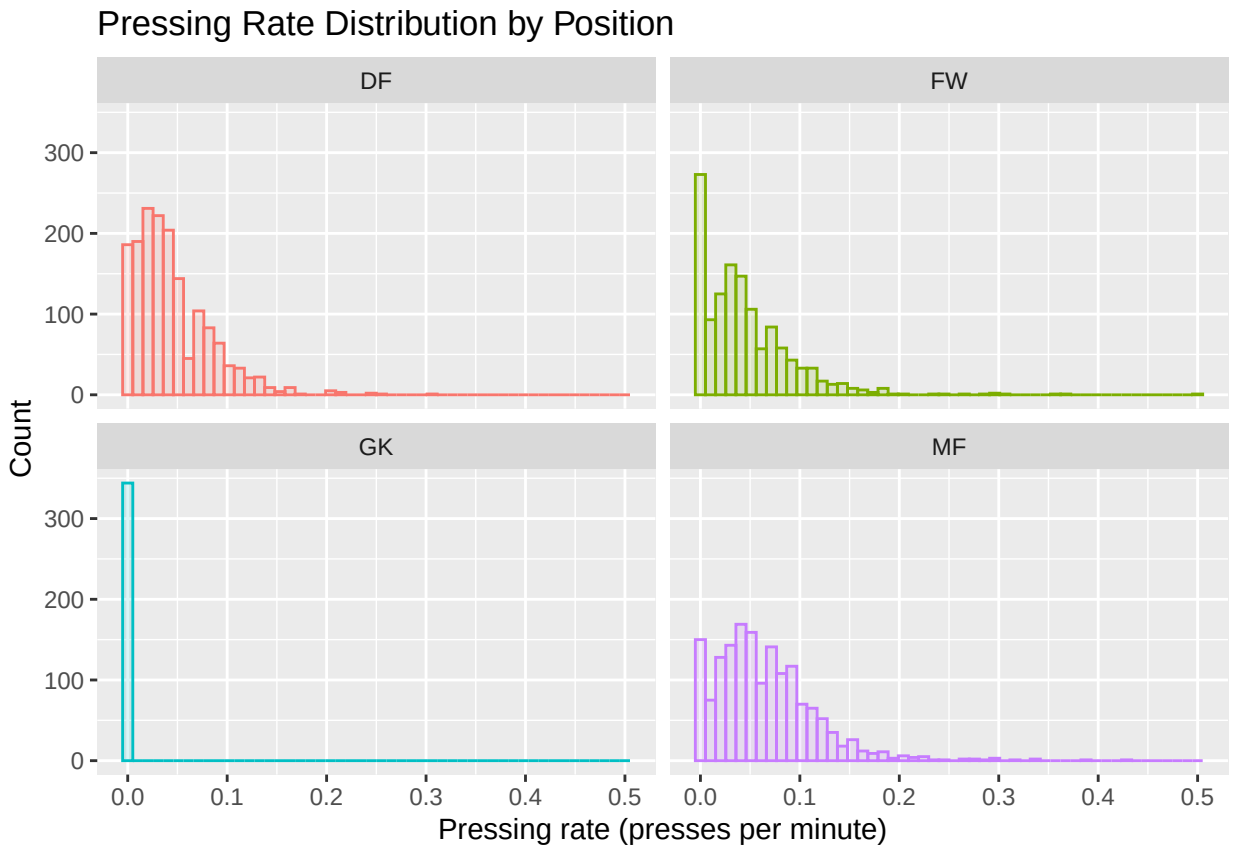

```r
# Within-Player Running Rate vs. Previous-Match Distance
df %>%
  filter(!is.na(prev_total_distance_m)) %>%
  group_by(player_tournament_id) %>%
  filter(n() >= 2) %>%
  mutate(
    prev_distance_dev = prev_total_distance_m - mean(prev_total_distance_m),
    running_rate_dev = normalized_total_distance_m - mean(normalized_total_distance_m)
  ) %>%
  ungroup() %>%
  ggplot(aes(
    x = prev_distance_dev,
    y = running_rate_dev,
    color = binned_minutes_played
  )) +
  geom_point(alpha = 0.4) +
  geom_smooth(method = "lm") +
  facet_wrap(~position) +
  labs(
    title = "Within-Player Running Rate vs. Previous-Match Distance",
    x = "Previous-match distance relative to player average (m)",
    y = "Running rate relative to player average (m/min)",
    color = "Minutes played"
  )
```

```
## `geom_smooth()` using formula = 'y ~ x'
```



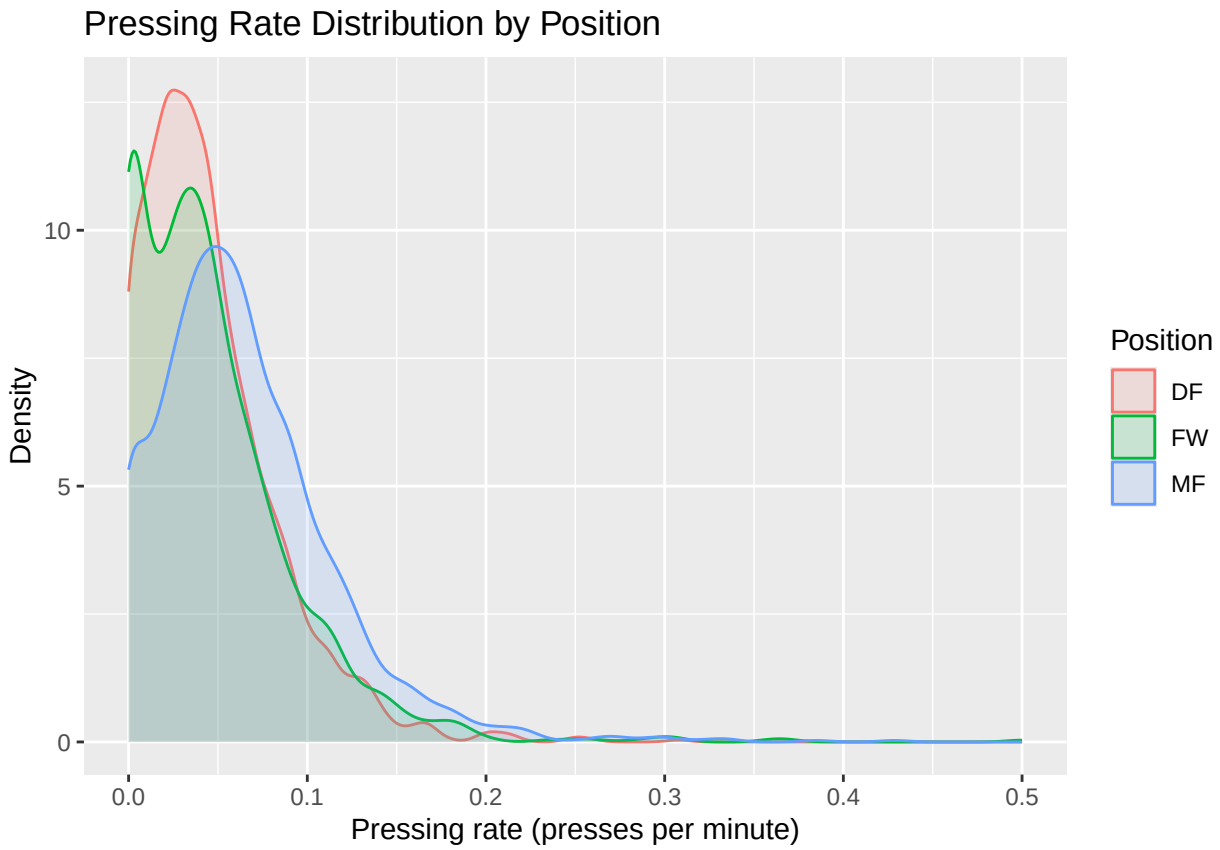
Pressing Rate

```
# Pressing Rate Distribution by Position
df %>%
  ggplot(aes(
    x = normalized_pressing_direct,
    color = position,
    fill = position
  )) +
  geom_histogram(alpha = 0.15, bins = 50) +
  facet_wrap(~position) +
  labs(
    title = "Pressing Rate Distribution by Position",
    x = "Pressing rate (presses per minute)",
    y = "Count",
    color = "Position",
    fill = "Position"
  ) +
  theme(legend.position = "none")
```

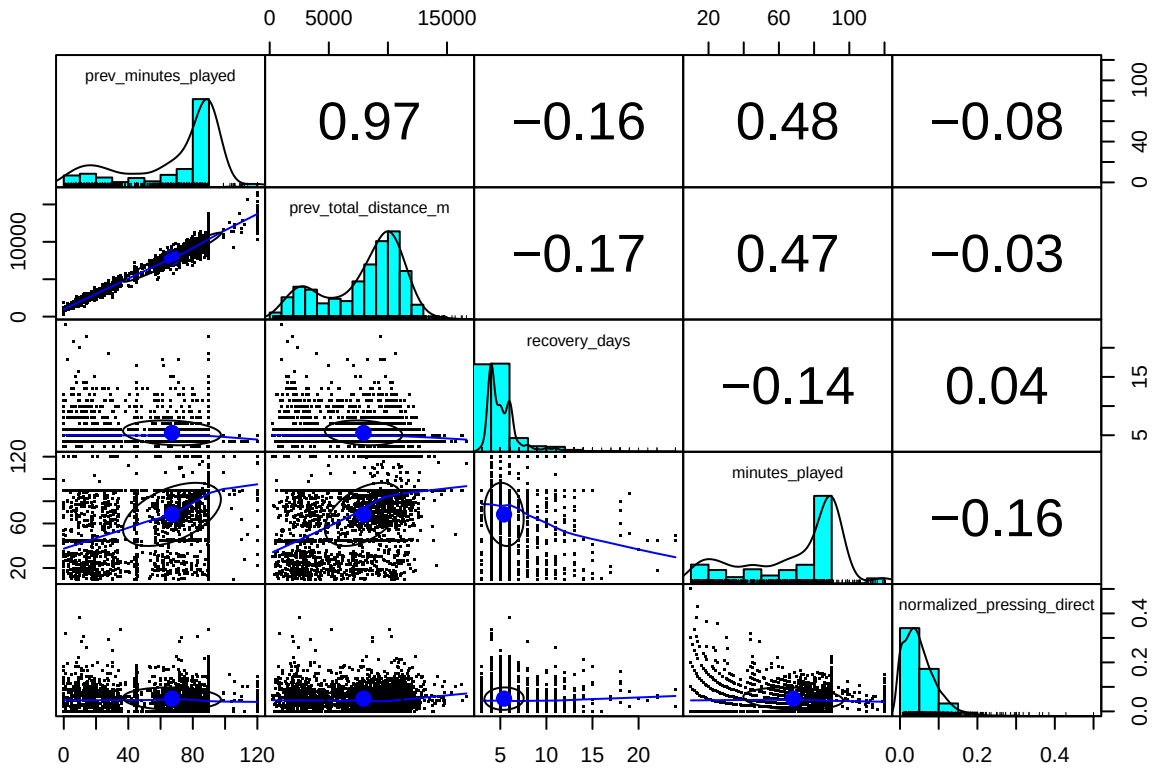



```r
# Remove all goalkeepers, since they all have 0 presses
df_no_gk <- df %>% filter(position != "GK")

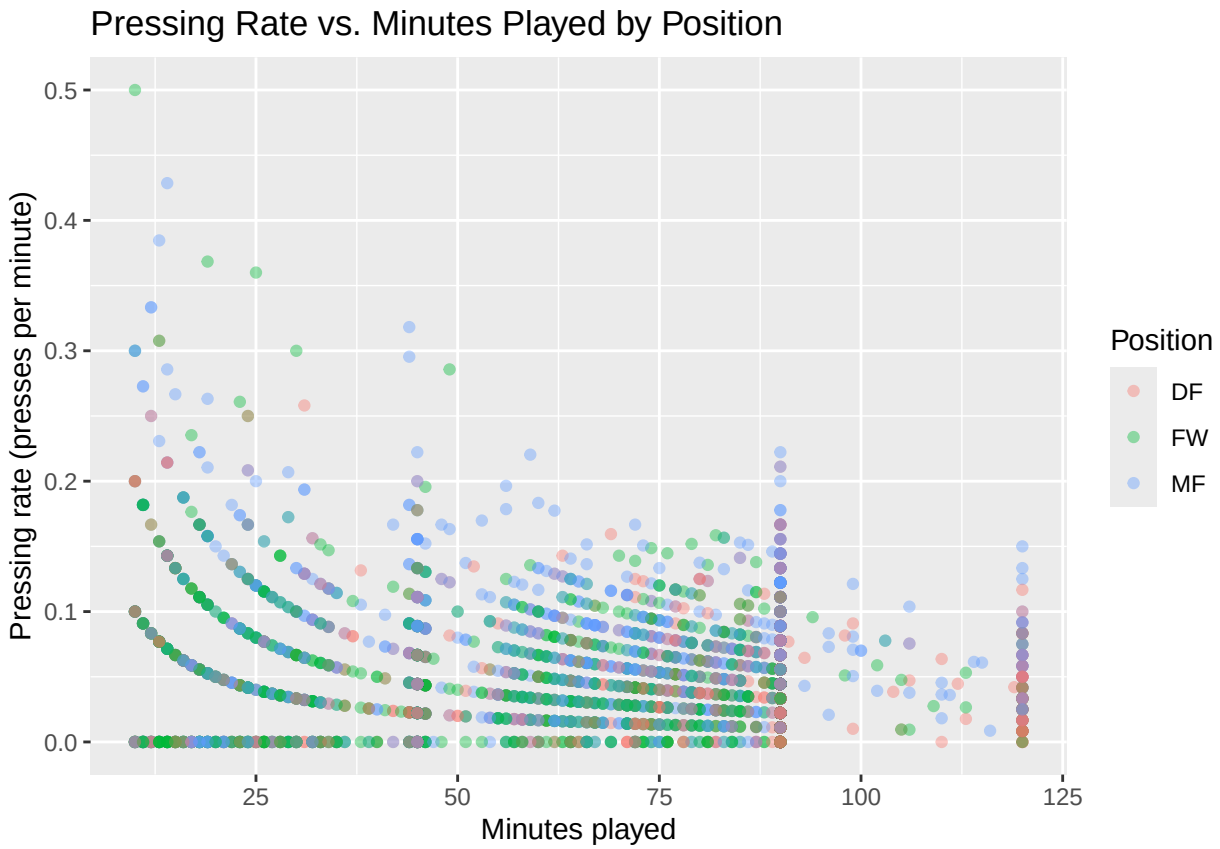
# Pressing Rate Distribution by Position
df_no_gk %>%
  ggplot(aes(
    x = normalized_pressing_direct,
    color = position,
    fill = position
  )) +
  geom_density(alpha = 0.15) +
  labs(
    title = "Pressing Rate Distribution by Position",
    x = "Pressing rate (presses per minute)",
    y = "Density",
    color = "Position",
    fill = "Position"
  )
```




```
pairs.panels(df_no_gk[, c(
  "prev_minutes_played",
  "prev_total_distance_m",
  "recovery_days",
  "minutes_played",
  "normalized_pressing_direct"
)], gap = 0, pch = ".")
```

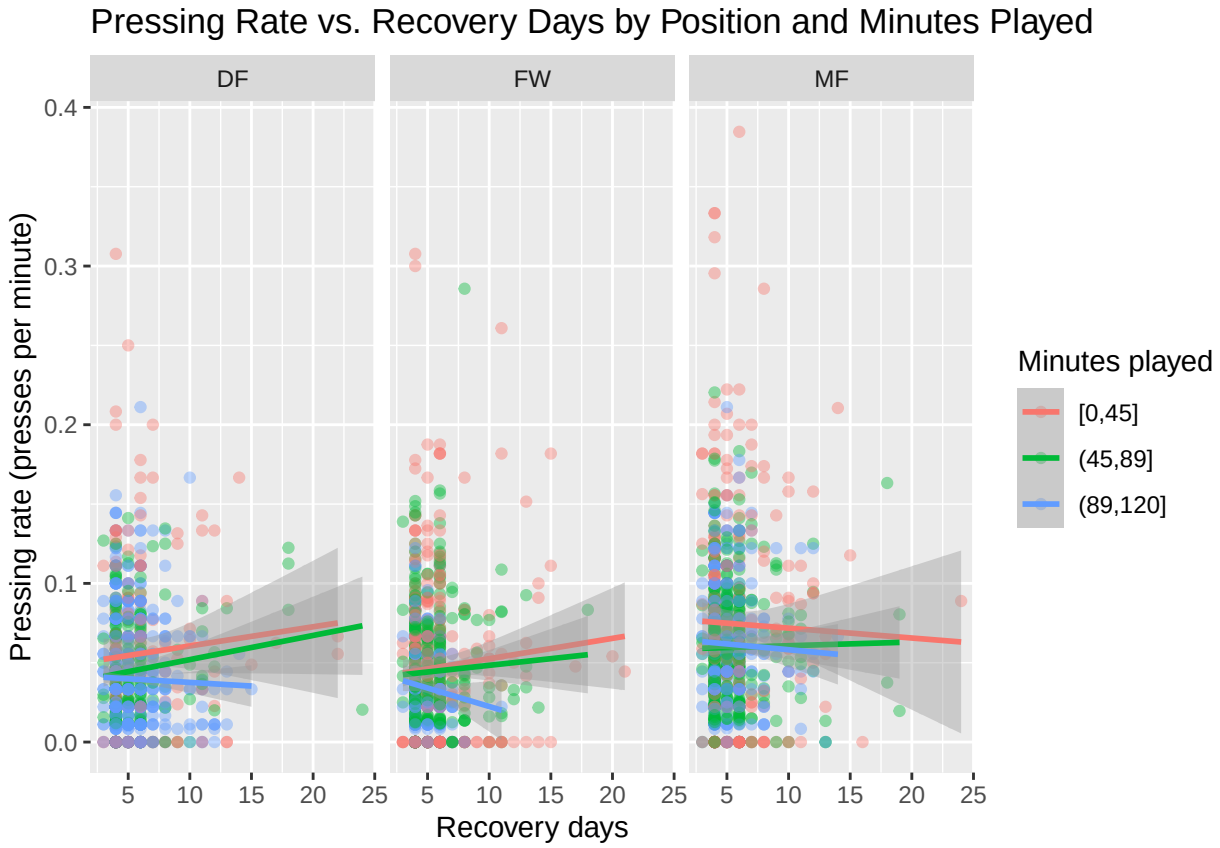



```r
# Pressing Rate vs. Minutes Played by Position
df_no_gk %>%
  ggplot(aes(
    x = minutes_played,
    y = normalized_pressing_direct,
    color = position
  )) +
  geom_point(alpha = 0.4) +
  labs(
    title = "Pressing Rate vs. Minutes Played by Position",
    x = "Minutes played",
    y = "Pressing rate (presses per minute)",
    color = "Position"
  )
```



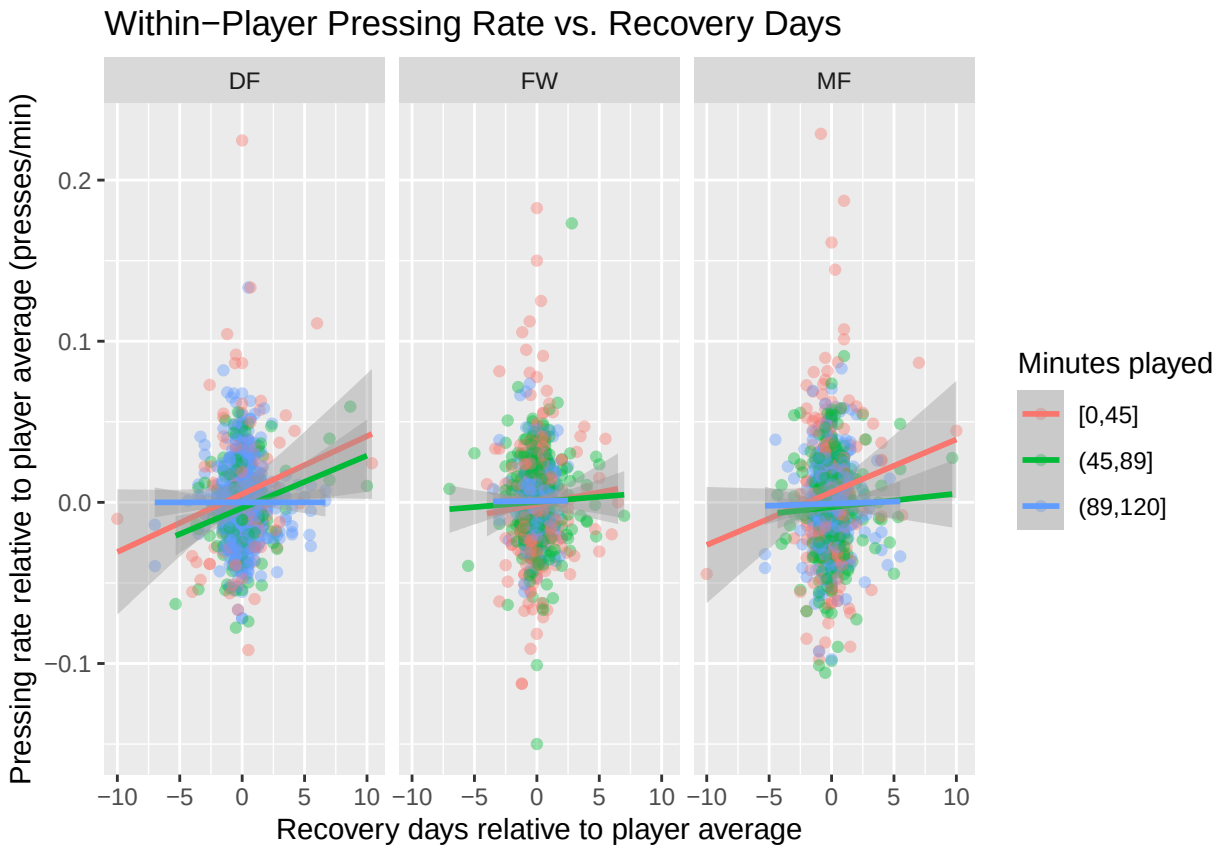

```
# Pressing Rate vs. Recovery Days by Position and Minutes Played
df_no_gk %>%
  filter(!is.na(recovery_days)) %>%
  # filter(recovery_days < 15) %>%
  ggplot(aes(
    x = recovery_days,
    y = normalized_pressing_direct,
    color = binned_minutes_played
  )) +
  geom_point(alpha = 0.4) +
  geom_smooth(method = "lm") +
  facet_wrap(~position) +
  labs(
    title = "Pressing Rate vs. Recovery Days by Position and Minutes Played",
    x = "Recovery days",
    y = "Pressing rate (presses per minute)",
    color = "Minutes played"
  )
```

```
## `geom_smooth()` using formula = 'y ~ x'
```



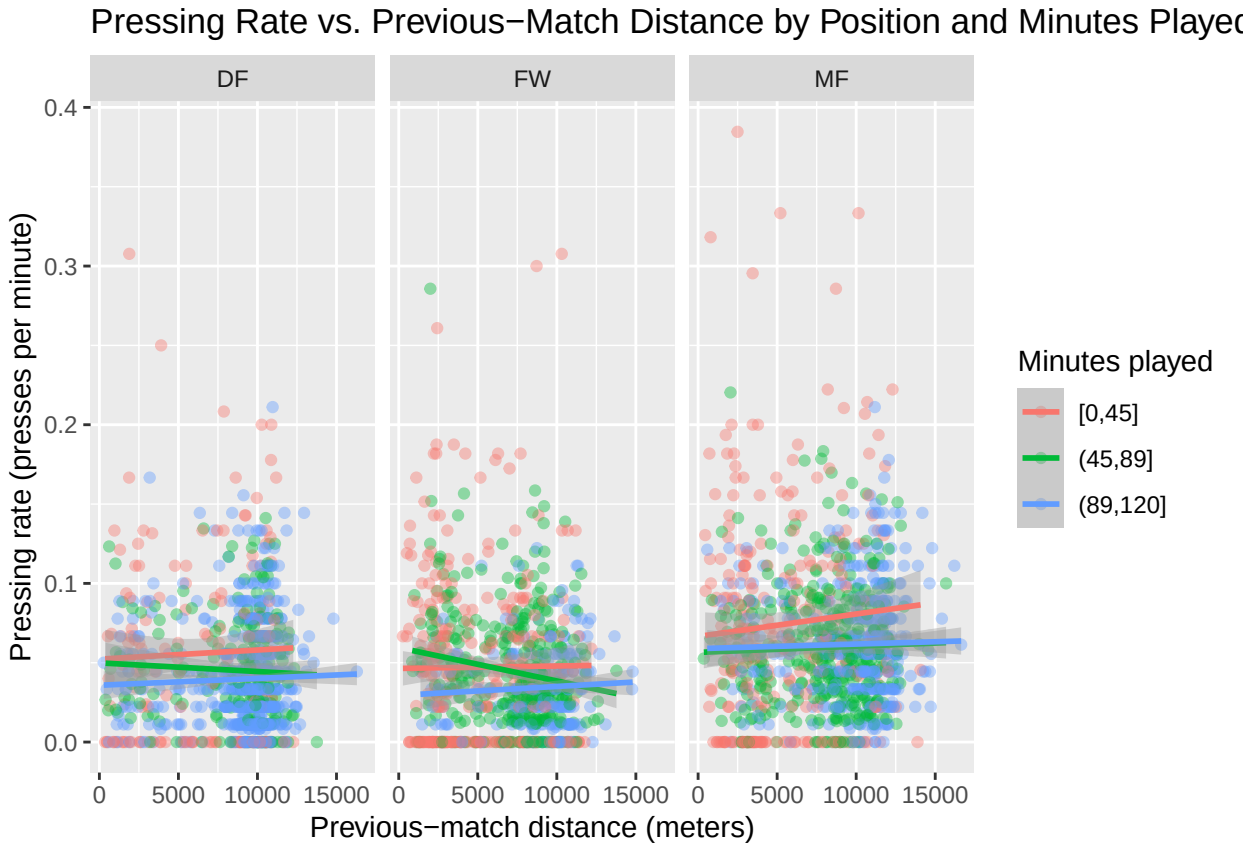

```
# Within-Player Pressing Rate vs. Recovery Days
df_no_gk %>%
  filter(!is.na(recovery_days)) %>%
  group_by(player_tournament_id) %>%
  filter(n() >= 2) %>%
  mutate(
    recovery_dev = recovery_days - mean(recovery_days),
    pressing_rate_dev = normalized_pressing_direct - mean(normalized_pressing_direct)
  ) %>%
  ungroup() %>%
  ggplot(aes(
    x = recovery_dev,
    y = pressing_rate_dev,
    color = binned_minutes_played
  )) +
  geom_point(alpha = 0.4) +
  geom_smooth(method = "lm") +
  facet_wrap(~position) +
  labs(
    title = "Within-Player Pressing Rate vs. Recovery Days",
    x = "Recovery days relative to player average",
    y = "Pressing rate relative to player average (presses/min)",
    color = "Minutes played"
  )
```

```
## `geom_smooth()` using formula = 'y ~ x'
```



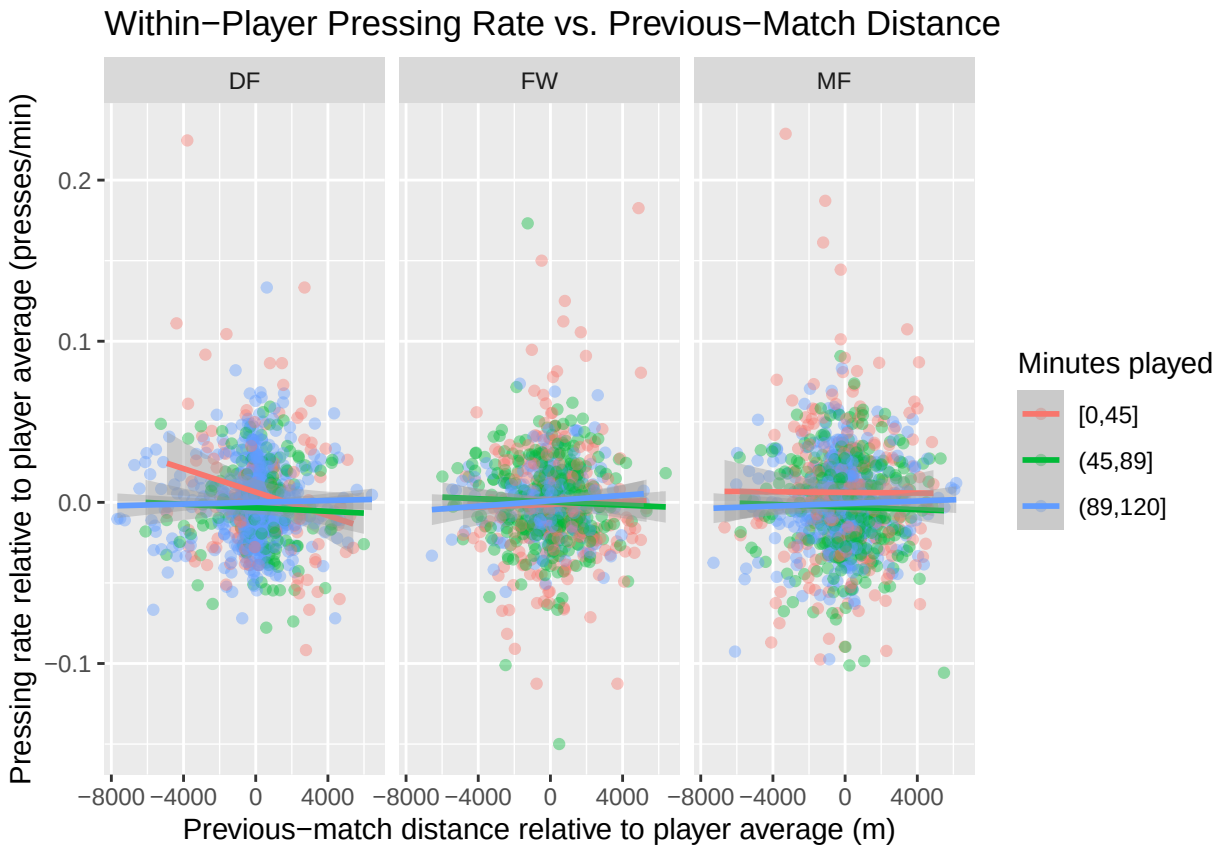

```r
# Pressing Rate vs. Previous-Match Distance by Position and Minutes Played
df_no_gk %>%
  filter(!is.na(prev_total_distance_m)) %>%
  ggplot(aes(
    x = prev_total_distance_m,
    y = normalized_pressing_direct,
    color = binned_minutes_played
  )) +
  geom_point(alpha = 0.4) +
  geom_smooth(method = "lm") +
  facet_wrap(~position) +
  labs(
    title = "Pressing Rate vs. Previous-Match Distance by Position and Minutes Played",
    x = "Previous-match distance (meters)",
    y = "Pressing rate (presses per minute)",
    color = "Minutes played"
  )
```

```
## `geom_smooth()` using formula = 'y ~ x'
```




```r
# Within-Player Pressing Rate vs. Previous-Match Distance
df_no_gk %>%
  filter(!is.na(prev_total_distance_m)) %>%
  group_by(player_tournament_id) %>%
  filter(n() >= 2) %>%
  mutate(
    prev_distance_dev = prev_total_distance_m - mean(prev_total_distance_m),
    pressing_rate_dev = normalized_pressing_direct - mean(normalized_pressing_direct)
  ) %>%
  ungroup() %>%
  ggplot(aes(
    x = prev_distance_dev,
    y = pressing_rate_dev,
    color = binned_minutes_played
  )) +
  geom_point(alpha = 0.4) +
  geom_smooth(method = "lm") +
  facet_wrap(~position) +
  labs(
    title = "Within-Player Pressing Rate vs. Previous-Match Distance",
    x = "Previous-match distance relative to player average (m)",
    y = "Pressing rate relative to player average (presses/min)",
    color = "Minutes played"
  )
```

```
## `geom_smooth()` using formula = 'y ~ x'
```



3. Modeling

Running Rate

```
# Ignore rows corresponding to first game at the tournament for a player (missing previous game data)
df_filtered <- df %>% filter(!is.na(recovery_days))
```

```
# normalized_total_distance_m ~ 1 + recovery_days + position
fit1 <- lm(normalized_total_distance_m ~
             recovery_days +
             position,
           data = df_filtered)
summary(fit1)
```

```
##
## Call:
## lm(formula = normalized_total_distance_m ~ recovery_days + position,
##     data = df_filtered)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -64.913 -10.599  -3.081   5.242 151.334
##
## Coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept)   108.2085     1.0111 107.024  < 2e-16 ***
## recovery_days   1.1170     0.1548   7.217 6.53e-13 ***
```

```
## positionFW      8.9868     0.8412  10.683  < 2e-16 ***
## positionGK    -59.1602     1.3307 -44.457  < 2e-16 ***
## positionMF     14.8891     0.7969  18.683  < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 18.76 on 3361 degrees of freedom
## Multiple R-squared:  0.4969, Adjusted R-squared:  0.4963
## F-statistic: 829.9 on 4 and 3361 DF,  p-value: < 2.2e-16
```

```r
# Add ns(minutes_played, df=4)
fit2 <- lm(normalized_total_distance_m ~
             recovery_days +
             position +
             ns(minutes_played, df=4),
           data = df_filtered)
summary(fit2)
```

```
##
## Call:
## lm(formula = normalized_total_distance_m ~ recovery_days + position +
##     ns(minutes_played, df = 4), data = df_filtered)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -80.910  -7.079   0.069   6.948 107.659
##
## Coefficients:
##                               Estimate Std. Error t value Pr(>|t|)
## (Intercept)                   165.2759     1.2026 137.432  < 2e-16 ***
## recovery_days                   0.3052     0.1066   2.863  0.00423 **
## positionFW                      2.6606     0.6105   4.358 1.35e-05 ***
## positionGK                    -55.9020     0.9136 -61.191  < 2e-16 ***
## positionMF                     12.1375     0.5598  21.682  < 2e-16 ***
## ns(minutes_played, df = 4)1  -55.4608     1.1423 -48.551  < 2e-16 ***
## ns(minutes_played, df = 4)2  -27.2774     1.0576 -25.793  < 2e-16 ***
## ns(minutes_played, df = 4)3 -118.3545     2.7727 -42.685  < 2e-16 ***
## ns(minutes_played, df = 4)4  -27.6551     1.1534 -23.977  < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 12.76 on 3357 degrees of freedom
## Multiple R-squared:  0.7675, Adjusted R-squared:  0.7669
## F-statistic:  1385 on 8 and 3357 DF,  p-value: < 2.2e-16
```

```r
# Add prev_total_distance_m
fit3 <- lm(normalized_total_distance_m ~
             recovery_days +
             position +
             prev_total_distance_m +
             ns(minutes_played, df=4),
           data = df_filtered)
summary(fit3)
```

```
##
```

```
## Call:
## lm(formula = normalized_total_distance_m ~ recovery_days + position +
##     prev_total_distance_m + ns(minutes_played, df = 4), data = df_filtered)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -81.614  -7.014   0.053   6.884 107.806
##
## Coefficients:
##                                Estimate Std. Error t value Pr(>|t|)
## (Intercept)                   1.644e+02  1.291e+00 127.323  < 2e-16 ***
## recovery_days                 3.276e-01  1.073e-01   3.054  0.00228 **
## positionFW                    2.787e+00  6.142e-01   4.537 5.92e-06 ***
## positionGK                   -5.526e+01  9.794e-01 -56.423  < 2e-16 ***
## positionMF                    1.212e+01  5.596e-01  21.664  < 2e-16 ***
## prev_total_distance_m         1.455e-04  8.009e-05   1.817  0.06936 .
## ns(minutes_played, df = 4)1 -5.592e+01  1.169e+00 -47.829  < 2e-16 ***
## ns(minutes_played, df = 4)2 -2.790e+01  1.112e+00 -25.100  < 2e-16 ***
## ns(minutes_played, df = 4)3 -1.193e+02  2.822e+00 -42.279  < 2e-16 ***
## ns(minutes_played, df = 4)4 -2.816e+01  1.187e+00 -23.735  < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 12.76 on 3356 degrees of freedom
## Multiple R-squared:  0.7677, Adjusted R-squared:  0.7671
## F-statistic:  1232 on 9 and 3356 DF,  p-value: < 2.2e-16
```

```
# Add random_effect(team_tournament_id)
fit4 <- lmer(normalized_total_distance_m ~
               recovery_days +
               position +
               prev_total_distance_m +
               ns(minutes_played, df=4) +
               (1 | team_tournament_id),
             data = df_filtered)
```

```
## Warning: Some predictor variables are on very different scales: consider rescaling.
## You may also use (g)lmerControl(autoscale = TRUE) to improve numerical stability.
## Warning: Some predictor variables are on very different scales: consider rescaling.
## You may also use (g)lmerControl(autoscale = TRUE) to improve numerical stability.
```

```
summary(fit4)
```

```
## Linear mixed model fit by REML. t-tests use Satterthwaite's method [
## lmerModLmerTest]
## Formula:
## normalized_total_distance_m ~ recovery_days + position + prev_total_distance_m +
##     ns(minutes_played, df = 4) + (1 | team_tournament_id)
##    Data: df_filtered
##
## REML criterion at convergence: 26576.6
##
## Scaled residuals:
##     Min      1Q  Median      3Q     Max
## -6.2877 -0.5314 -0.0013  0.5115  8.3995
```

```
##
## Random effects:
##  Groups              Name        Variance Std.Dev.
##  team_tournament_id (Intercept)  11.49    3.389
##  Residual                        152.28   12.340
## Number of obs: 3366, groups:  team_tournament_id, 80
##
## Fixed effects:
##                                Estimate Std. Error         df t value Pr(>|t|)
## (Intercept)                   1.642e+02  1.336e+00  2.289e+03 122.971  < 2e-16
## recovery_days                 3.945e-01  1.103e-01  3.298e+03   3.577 0.000352
## positionFW                    2.660e+00  6.000e-01  3.313e+03   4.433  9.6e-06
## positionGK                   -5.546e+01  9.492e-01  3.285e+03 -58.429  < 2e-16
## positionMF                    1.219e+01  5.456e-01  3.307e+03  22.349  < 2e-16
## prev_total_distance_m         1.091e-04  7.794e-05  3.302e+03   1.400 0.161541
## ns(minutes_played, df = 4)1 -5.583e+01  1.134e+00  3.288e+03 -49.224  < 2e-16
## ns(minutes_played, df = 4)2 -2.776e+01  1.085e+00  3.313e+03 -25.590  < 2e-16
## ns(minutes_played, df = 4)3 -1.181e+02  2.772e+00  3.332e+03 -42.600  < 2e-16
## ns(minutes_played, df = 4)4 -2.735e+01  1.178e+00  3.346e+03 -23.222  < 2e-16
##
## (Intercept)                  ***
## recovery_days                ***
## positionFW                   ***
## positionGK                   ***
## positionMF                   ***
## prev_total_distance_m
## ns(minutes_played, df = 4)1 ***
## ns(minutes_played, df = 4)2 ***
## ns(minutes_played, df = 4)3 ***
## ns(minutes_played, df = 4)4 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Correlation of Fixed Effects:
##             (Intr) rcvry_ pstnFW pstnGK pstnMF prv___ n(_,d=4)1 n(_,d=4)2
## recovry_dys -0.546
## positionFW  -0.353  0.060
## positionGK  -0.209  0.065  0.244
## positionMF  -0.260  0.033  0.504  0.234
## prv_ttl_ds_ -0.344  0.113  0.116  0.364 -0.013
## ns(m_,d=4)1 -0.289  0.056 -0.044 -0.067 -0.040 -0.215
## ns(m_,d=4)2 -0.325  0.067  0.268 -0.213  0.211 -0.307 -0.125
## ns(m_,d=4)3 -0.569  0.030  0.114 -0.113  0.064 -0.189  0.225     0.586
## ns(m_,d=4)4 -0.076  0.045  0.115 -0.153  0.105 -0.234  0.312     0.156
##             n(_,d=4)3
## recovry_dys
## positionFW
## positionGK
## positionMF
## prv_ttl_ds_
## ns(m_,d=4)1
## ns(m_,d=4)2
## ns(m_,d=4)3
## ns(m_,d=4)4  0.194
```

```
## fit warnings:
## Some predictor variables are on very different scales: consider rescaling.
## You may also use (g)lmerControl(autoscale = TRUE) to improve numerical stability.
```

```
# Add random_effect(player_tournament_id)
fit5 <- lmer(normalized_total_distance_m ~
               recovery_days +
               position +
               prev_total_distance_m +
               ns(minutes_played, df=4) +
               (1 | team_tournament_id) +
               (1 | player_tournament_id),
             data = df_filtered)
```

```
## Warning: Some predictor variables are on very different scales: consider rescaling.
## You may also use (g)lmerControl(autoscale = TRUE) to improve numerical stability.
## Warning: Some predictor variables are on very different scales: consider rescaling.
## You may also use (g)lmerControl(autoscale = TRUE) to improve numerical stability.
```

```
summary(fit5)
```

```
## Linear mixed model fit by REML. t-tests use Satterthwaite's method [
## lmerModLmerTest]
## Formula:
## normalized_total_distance_m ~ recovery_days + position + prev_total_distance_m +
##     ns(minutes_played, df = 4) + (1 | team_tournament_id) + (1 |
##     player_tournament_id)
##    Data: df_filtered
##
## REML criterion at convergence: 26105.6
##
## Scaled residuals:
##     Min      1Q  Median      3Q     Max
## -6.0030 -0.4319 -0.0075  0.4111  7.2063
##
## Random effects:
##  Groups               Name        Variance Std.Dev.
##  player_tournament_id (Intercept) 72.855   8.536
##  team_tournament_id   (Intercept)  9.629   3.103
##  Residual                         87.694   9.364
## Number of obs: 3366, groups:
## player_tournament_id, 1339; team_tournament_id, 80
##
## Fixed effects:
##                                Estimate Std. Error         df t value Pr(>|t|)
## (Intercept)                   1.644e+02  1.343e+00  2.222e+03 122.486  < 2e-16
## recovery_days                 3.247e-01  1.001e-01  3.180e+03   3.244  0.00119
## positionFW                    3.295e+00  8.021e-01  1.230e+03   4.107 4.27e-05
## positionGK                   -5.795e+01  1.282e+00  1.147e+03 -45.203  < 2e-16
## positionMF                    1.247e+01  7.365e-01  1.162e+03  16.932  < 2e-16
## prev_total_distance_m        -1.956e-04  7.222e-05  3.227e+03  -2.709  0.00679
## ns(minutes_played, df = 4)1 -5.399e+01  1.041e+00  3.075e+03 -51.870  < 2e-16
## ns(minutes_played, df = 4)2 -2.458e+01  9.982e-01  3.107e+03 -24.626  < 2e-16
## ns(minutes_played, df = 4)3 -1.122e+02  2.499e+00  3.080e+03 -44.875  < 2e-16
## ns(minutes_played, df = 4)4 -2.538e+01  1.000e+00  2.610e+03 -25.372  < 2e-16
```

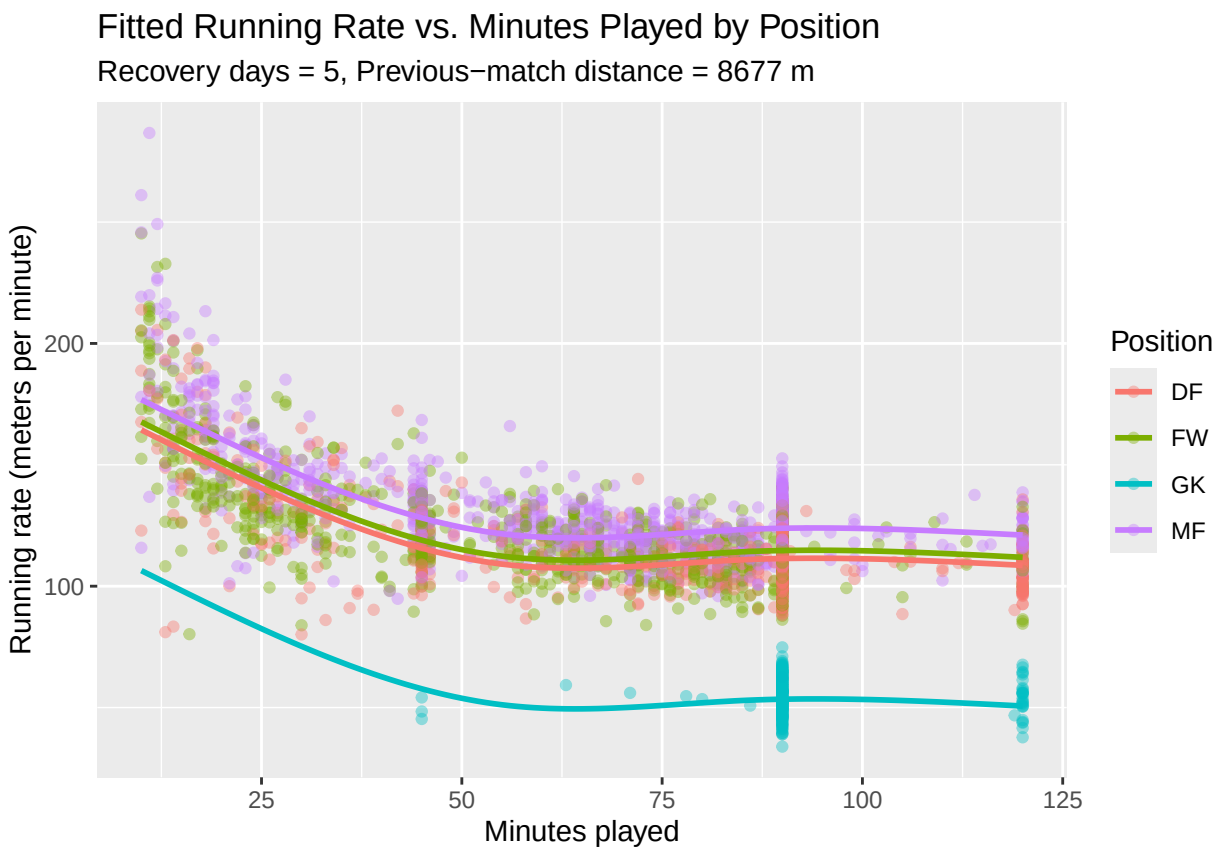
```
##
## (Intercept)                 ***
## recovery_days               **
## positionFW                  ***
## positionGK                  ***
## positionMF                  ***
## prev_total_distance_m       **
## ns(minutes_played, df = 4)1 ***
## ns(minutes_played, df = 4)2 ***
## ns(minutes_played, df = 4)3 ***
## ns(minutes_played, df = 4)4 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Correlation of Fixed Effects:
##             (Intr) rcvry_ pstnFW pstnGK pstnMF prv___ n(_,d=4)1 n(_,d=4)2
## recovry_dys -0.505
## positionFW  -0.406  0.035
## positionGK  -0.226  0.051  0.261
## positionMF  -0.335  0.020  0.492  0.269
## prv_ttl_ds_ -0.417  0.121  0.146  0.229  0.044
## ns(m_,d=4)1 -0.320  0.048  0.006 -0.023 -0.017 -0.108
## ns(m_,d=4)2 -0.363  0.063  0.222 -0.119  0.158 -0.132 -0.124
## ns(m_,d=4)3 -0.558  0.028  0.101 -0.061  0.047 -0.097  0.250     0.587
## ns(m_,d=4)4 -0.136  0.030  0.118 -0.076  0.088 -0.103  0.312     0.226
##             n(_,d=4)3
## recovry_dys
## positionFW
## positionGK
## positionMF
## prv_ttl_ds_
## ns(m_,d=4)1
## ns(m_,d=4)2
## ns(m_,d=4)3
## ns(m_,d=4)4  0.197
## fit warnings:
## Some predictor variables are on very different scales: consider rescaling.
## You may also use (g)lmerControl(autoscale = TRUE) to improve numerical stability.
```

```
final_running_rate_fit = fit5
```

```
# Plot spline curve for minutes_played by fixing recovery days and previous-match distance to median va
spline_grid <- expand.grid(
  minutes_played = seq(min(df_filtered$minutes_played), max(df_filtered$minutes_played), by = 1),
  position = sort(unique(df_filtered$position)),
  recovery_days = median(df_filtered$recovery_days),
  prev_total_distance_m = median(df_filtered$prev_total_distance_m)
)
spline_grid$fitted <- predict(final_running_rate_fit, newdata = spline_grid, re.form = NA)

# Overlay fitted curves on the observed data
df_filtered %>%
  ggplot(aes(
    x = minutes_played,
    y = normalized_total_distance_m,
```

```
    color = position
  )) +
  geom_point(alpha = 0.4) +
  geom_line(data = spline_grid, aes(y = fitted), linewidth = 1) +
  labs(
    title = "Fitted Running Rate vs. Minutes Played by Position",
    subtitle = paste0(
      "Recovery days = ", spline_grid$recovery_days[1],
      ", Previous-match distance = ", round(spline_grid$prev_total_distance_m[1]), " m"
    ),
    x = "Minutes played",
    y = "Running rate (meters per minute)",
    color = "Position"
  )
```




```
# Extract random intercepts and their conditional standard errors
running_rate_re <- as.data.frame(ranef(final_running_rate_fit, condVar = TRUE)) %>%
  rename(intercept = condval, se = condsd)

# Team random intercepts, with team name and tournament year attached
team_info <- df_filtered %>%
  distinct(team_tournament_id, team, tournament_year)

team_re <- running_rate_re %>%
  filter(grpvar == "team_tournament_id") %>%
  transmute(team_tournament_id = as.character(grp), intercept, se) %>%
  left_join(team_info, by = "team_tournament_id") %>%
```

```r
  arrange(desc(intercept))

# Player random intercepts, with player name, team, and position attached
player_info <- df_filtered %>%
  distinct(player_tournament_id, player_name, team, tournament_year, position)

player_re <- running_rate_re %>%
  filter(grpvar == "player_tournament_id") %>%
  transmute(player_tournament_id = as.character(grp), intercept, se) %>%
  left_join(player_info, by = "player_tournament_id") %>%
  arrange(desc(intercept))
```

```r
# Top/bottom 10 team random intercepts
head(team_re, 10)
```

```
##    team_tournament_id intercept       se        team tournament_year
## 1           2022_USA  6.784539 2.019945         USA             2022
## 2        2022_IR_IRAN  6.285863 2.071050     IR Iran             2022
## 3         2022_SERBIA  4.948113 2.117504      Serbia             2022
## 4      2022_AUSTRALIA  4.811341 2.019945   Australia             2022
## 5           2026_USA  4.382351 1.958467         USA             2026
## 6         2026_MEXICO  4.117611 1.899432      Mexico             2026
## 7        2026_TURKIYE  3.574803 2.125064     Türkiye             2026
## 8        2022_URUGUAY  3.307946 2.088541     Uruguay             2022
## 9    2026_NEW_ZEALAND  3.273544 2.057277 New Zealand             2026
## 10       2026_BELGIUM  3.089291 1.833344     Belgium             2026
```

```r
tail(team_re, 10)
```

```
##    team_tournament_id intercept       se          team tournament_year
## 71       2026_CURACAO -2.796069 2.171273       Curaçao             2026
## 72         2026_JAPAN -2.807045 1.975635         Japan             2026
## 73 2026_COTE_D_IVOIRE -2.969030 2.025490 Côte d'Ivoire             2026
## 74   2026_NETHERLANDS -3.021554 1.996484   Netherlands             2026
## 75    2026_CABO_VERDE -3.117190 1.916607    Cabo Verde             2026
## 76       2026_ECUADOR -3.175865 2.036162       Ecuador             2026
## 77         2026_QATAR -3.188940 2.081524         Qatar             2026
## 78       2022_ECUADOR -3.323324 2.191452       Ecuador             2022
## 79      2026_COLOMBIA -3.513049 1.947492      Colombia             2026
## 80         2026_GHANA -4.669574 1.929695         Ghana             2026
```

```r
# Top/bottom 10 player random intercepts
player_re %>% select(player_name, team, tournament_year, position, intercept, se) %>% head(10)
```

```
##           player_name      team tournament_year position intercept       se
## 1  KYEREH Daniel-Kofi     Ghana            2022       MF  37.32491 5.393558
## 2      Alvaro FIDALGO    Mexico            2026       MF  36.91633 5.364607
## 3          GOMEZ Maxi   Uruguay            2022       FW  33.37241 6.379082
## 4   PALACIOS Exequiel Argentina            2022       MF  28.66184 5.354111
## 5      Orbelin PINEDA    Mexico            2026       MF  27.89513 6.366897
## 6      DJURICIC Filip    Serbia            2022       FW  27.64837 6.381048
## 7      Thomas MEUNIER   Belgium            2026       DF  21.72400 5.355639
## 8   Jacob SHAFFELBURG    Canada            2026       MF  21.51444 4.763929
## 9     Gonzalo MONTIEL Argentina            2026       DF  20.30923 4.018364
## 10     BERNARDO SILVA  Portugal            2026       MF  19.56536 5.369925
```

```r
player_re %>% select(player_name, team, tournament_year, position, intercept, se) %>% tail(10)
```

```
##              player_name      team tournament_year position intercept       se
## 1330       AZMOUN Sardar   IR Iran            2022       FW -17.91703 5.389302
## 1331       MBAPPE Kylian    France            2022       FW -17.94791 3.823945
## 1332       GORETZKA Leon   Germany            2022       MF -18.04729 5.400983
## 1333       AYMEN HUSSEIN      Iraq            2026       FW -18.75792 6.384216
## 1334        Malick THIAW   Germany            2026       DF -19.38609 6.371343
## 1335         AHMED QASEM      Iraq            2026       FW -19.79187 5.403422
## 1336       Romelu LUKAKU   Belgium            2026       FW -20.79268 4.340316
## 1337        Lionel MESSI Argentina            2026       FW -23.80367 3.597786
## 1338        Nadiem AMIRI   Germany            2026       MF -28.95883 6.371343
## 1339 MOHAMED ABDELMONEIM     Egypt            2026       DF -34.68945 6.366299
```

```r
# Famous players to always show in the dot and whisker plot (if present in the model)
famous_players <- c(
  # FW
  "2022_ARGENTINA_MESSI_LIONEL_10",
  "2026_ARGENTINA_LIONEL_MESSI_10",
  "2026_PORTUGAL_CRISTIANO_RONALDO_7",
  "2026_FRANCE_KYLIAN_MBAPPE_10",
  "2026_ENGLAND_HARRY_KANE_9",
  "2026_NORWAY_ERLING_HAALAND_9",
  "2022_BRAZIL_NEYMAR_10",
  # MF
  "2022_CROATIA_MODRIC_LUKA_10",
  "2026_BELGIUM_KEVIN_DE_BRUYNE_7",
  "2026_SPAIN_PEDRI_20",
  "2022_ENGLAND_BELLINGHAM_JUDE_22",
  "2026_GERMANY_JAMAL_MUSIALA_10",
  # DF
  "2026_NETHERLANDS_VIRGIL_VAN_DIJK_4",
  "2022_MOROCCO_HAKIMI_ACHRAF_2",
  "2022_CROATIA_GVARDIOL_JOSKO_20",
  "2026_BRAZIL_MARQUINHOS_4",
  # GK
  "2022_GERMANY_NEUER_MANUEL_1",
  "2026_BELGIUM_THIBAUT_COURTOIS_1",
  "2022_ARGENTINA_MARTINEZ_EMILIANO_23",
  "2026_BRAZIL_ALISSON_1"
)

# Famous teams to always show in the team dot and whisker plot
famous_teams <- c("Argentina", "Brazil", "France", "England", "Germany", "Spain", "Portugal")

# Number of evenly spaced players/teams (by intercept rank) to show per panel
n_spread <- 4

player_re_subset <- player_re %>%
  mutate(famous = player_tournament_id %in% famous_players) %>%
  group_by(position) %>%
  arrange(intercept, .by_group = TRUE) %>%
  filter(famous | row_number() %in% round(seq(1, n(), length.out = n_spread))) %>%
  ungroup() %>%
```

```r
  mutate(label = paste0(player_name, " (", team, " ", tournament_year, ")"))

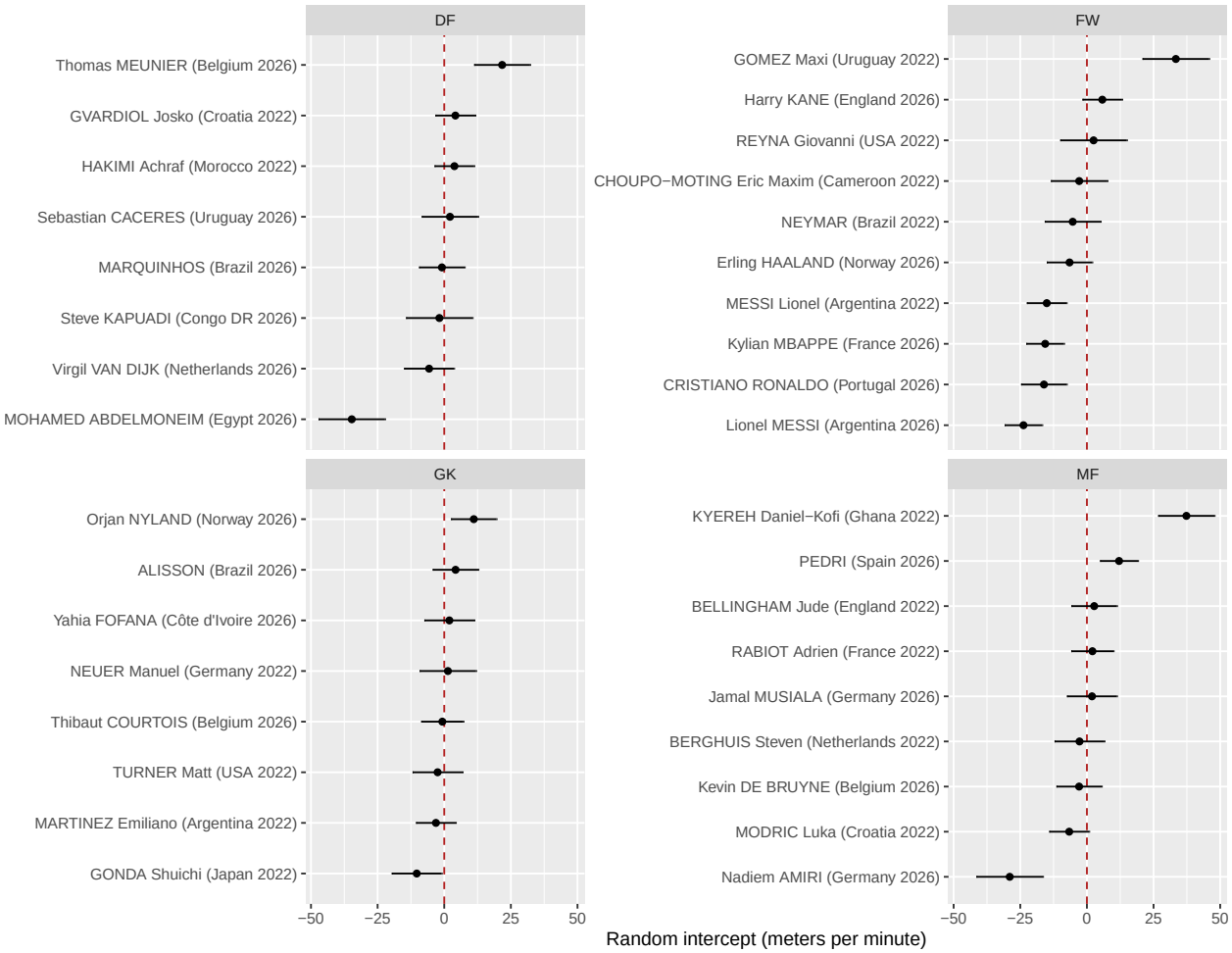
team_re_subset <- team_re %>%
  mutate(famous = team %in% famous_teams) %>%
  group_by(tournament_year) %>%
  arrange(intercept, .by_group = TRUE) %>%
  filter(famous | row_number() %in% round(seq(1, n(), length.out = n_spread))) %>%
  ungroup()
```

```r
# Dot and whisker plot of player random intercepts (95% intervals) by position
player_re_subset %>%
  ggplot(aes(
    x = intercept,
    y = reorder(label, intercept)
  )) +
  geom_vline(xintercept = 0, linetype = "dashed", color = "firebrick") +
  geom_errorbarh(aes(xmin = intercept - 1.96 * se, xmax = intercept + 1.96 * se), height = 0) +
  geom_point() +
  facet_wrap(~position, scales = "free_y") +
  labs(
    title = "Player Random Intercepts for Running Rate by Position",
    x = "Random intercept (meters per minute)",
    y = NULL
  ) +
  theme(legend.position = "none", plot.title.position = "plot")
```

```
## Warning: `geom_errorbarh()` was deprecated in ggplot2 4.0.0.
## i Please use the `orientation` argument of `geom_errorbar()` instead.
## This warning is displayed once per session.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```

```
## `height` was translated to `width`.
```

Player Random Intercepts for Running Rate by Position




```
# Dot and whisker plot of team random intercepts (95% intervals) by tournament year
team_re_subset %>%
  ggplot(aes(
    x = intercept,
    y = reorder(team_tournament_id, intercept)
  )) +
  geom_vline(xintercept = 0, linetype = "dashed", color = "firebrick") +
  geom_errorbarh(aes(xmin = intercept - 1.96 * se, xmax = intercept + 1.96 * se), height = 0) +
  geom_point() +
  facet_wrap(~tournament_year, scales = "free_y") +
  scale_y_discrete(labels = function(x) team_info$team[match(x, team_info$team_tournament_id)]) +
  labs(
    title = "Team Random Intercepts for Running Rate by Tournament",
    x = "Random intercept (meters per minute)",
    y = NULL
  ) +
  theme(legend.position = "none", plot.title.position = "plot")
```

```
## `height` was translated to `width`.
```

Team Random Intercepts for Running Rate by Tournament

